

untangle::async

Generated by Doxygen 1.18.0

1	async	1
2	Topic Index	5
2.1	Topics	5
3	Namespace Index	7
3.1	Namespace List	7
4	Class Index	9
4.1	Class List	9
5	File Index	11
5.1	File List	11
6	Topic Documentation	13
6.1	namespace untangle: functions	13
7	Namespace Documentation	15
7.1	untangle Namespace Reference	15
7.1.1	Detailed Description	15
8	Class Documentation	17
8.1	untangle::async::attachment Struct Reference	17
8.1.1	Detailed Description	17
8.2	untangle::async::execution< actionT > Class Template Reference	17
8.2.1	Detailed Description	18
8.2.2	Constructor & Destructor Documentation	19
8.2.2.1	execution() [1/2]	19
8.2.2.2	execution() [2/2]	19
8.2.2.3	~execution()	19
8.2.3	Member Function Documentation	20
8.2.3.1	create_instance()	20
8.2.3.2	bind_action_and_method()	20
8.2.3.3	bind_action_and_function()	21
8.2.3.4	is_running()	21
8.2.3.5	stop()	21
8.2.3.6	add_action()	22
8.2.3.7	attach()	22
8.2.3.8	detach()	23
8.3	untangle::async::execution_poll Class Reference	23
8.3.1	Detailed Description	24
8.3.2	Member Function Documentation	24
8.3.2.1	add()	24
8.3.2.2	remove()	24
8.3.2.3	is_running()	25
8.3.2.4	get()	25

8.4 untangle::async::invalid_attachment Struct Reference	25
8.4.1 Detailed Description	26
8.4.2 Constructor & Destructor Documentation	26
8.4.2.1 invalid_attachment()	26
8.4.3 Member Function Documentation	26
8.4.3.1 what()	26
9 File Documentation	27
9.1 async.hpp	27
Index	33

Chapter 1

async

c++ mechanism to run queued callables in an asynchronous fashion

The interface consists of one header file that exposes a generic execution class. It is a wrapper of a list of callables that are invoked on other thread. The callables are pumped into the list by invoking an action from the caller thread.

The implementation is using the **actuator** callable, and therefore it exemplifies how a generic callable may improve the code structure, by easily creating interfaces inside a class instead of using external ones.

There are provided two execution modes: one-off and continuous.

requirements

C++23. The interface reports warnings with `std::println`, so it needs a standard library that provides `<print>` — GCC 14 or newer; older `libc++` and `MSVC` releases may not have it yet.

one-off execution



example: one-off execution of a plain function

```
#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer,
    // because an async binding keeps only weak ownership of it
    // the name is optional, and identifies the execution in the warnings it reports
    using void_execution = untangle::async::execution<std::function<void(void)>>;
    auto execution = void_execution::create_instance("oneoff");
```

```

std::function<void(void)> action;
// create async binding between the action and f; the execution is passed as a shared pointer,
// which the binding holds weakly - that is what lets the action outlive it safely
void_execution::bind_action_and_function(action, f, execution);
// whenever the action is invoked, a new callable wrapping f will be added to the execution's action list
action();
// run the execution
execution->run();

// add the execution object to the running poll
untangle::async::execution_poll::get().add(*execution);

// wait the polled executions to finish
while(untangle::async::execution_poll::get().is_running())
{
    std::this_thread::sleep_for(std::chrono::milliseconds(1));
}

return 0;
}

```

example: one-off execution of a class method

```

#include <async.hpp>

class A
{
public:
    void f(int x)
    {
        std::cout << "A::f " << x << " thread " << std::this_thread::get_id() << std::endl;
    }
    std::function<void(int)> action;
};

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    auto a = std::make_shared<A>();

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using int_execution = untangle::async::execution<std::function<void(int)>>;
    auto execution = int_execution::create_instance("oneoff_method");
    // note both the bound object and the execution must be shared pointers
    int_execution::bind_action_and_method(a->action, a, &A::f, execution);
    // whenever the action is invoked, a new callable wrapping f will be added to the execution's action list
    a->action(10);
    // run the execution
    execution->run();

    // add the execution object to the running poll
    untangle::async::execution_poll::get().add(*execution);

    // wait the polled executions to finish
    while(untangle::async::execution_poll::get().is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

continuous execution



example: continuous execution of a plain function

```

#include <async.hpp>

void f()
{
    std::cout << "f thread " << std::this_thread::get_id() << std::endl;
}

```

```

}

int main()
{
    std::cout << "main thread " << std::this_thread::get_id() << std::endl;

    // declare an execution specialized for an action type; it must be held by a shared pointer
    using void_execution = untangle::async::execution<std::function<void(void)>>;
    auto execution = void_execution::create_instance("continuous");
    std::function<void(void)> action;
    // create async binding between the action and f
    void_execution::bind_action_and_function(action, f, execution);

    // run the execution
    execution->start();
    // add f to the execution's queue (three times)
    action();action();action();
    // stop the execution
    execution->stop();

    // add the execution object to the running poll
    untangle::async::execution_poll::get().add(*execution);

    // wait the polled executions to finish
    while(untangle::async::execution_poll::get().is_running())
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(1));
    }

    return 0;
}

```

queueing and lifetime

Queueing is thread safe. An action may be invoked from any thread; the execution's list is guarded, and the worker is woken rather than polled for.

`stop()` ends the execution's working life. The worker runs whatever was queued before the call and then leaves. An action invoked after `stop()` is refused rather than queued, because it would never run, and the refusal is reported:

```
warning: execution 'continuous' is stopped, action not added
```

An action bound to an object that has since been destroyed is dropped the same way, rather than ending the process:

```
warning: execution 'continuous' dropped an invalid action: bind: invalid object
```

The worker is detached and cannot be joined. Waiting for an execution therefore means waiting on its running state, which is what `execution_poll` reports and what the examples above do. The destructor waits on the same state, so an execution that goes out of scope while its worker is still running blocks until the worker is finished rather than leaving it reading freed memory.

The poll holds a pointer to each execution added to it, so an execution withdraws itself in its destructor. `execution_poll::remove()` is available for withdrawing one earlier; calling it is not required.

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

namespace untangle: functions [13](#)

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	Interface to untangle::async::execution class	15
--------------------------	---	--------------------

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::async::attachment	
One execution's place in the attachment graph: who attached it, if anyone	17
untangle::async::execution< actionT >	
Asynchronous template execution class	17
untangle::async::execution_poll	
Execution poll class	23
untangle::async::invalid_attachment	
Invalid attachment exception	25

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

async.hpp		27
---------------------------	-------	----

Chapter 6

Topic Documentation

6.1 namespace untangle: functions

Chapter 7

Namespace Documentation

7.1 untangle Namespace Reference

Interface to [untangle::async::execution](#) class.

7.1.1 Detailed Description

Interface to [untangle::async::execution](#) class.

Chapter 8

Class Documentation

8.1 `untangle::async::attachment` Struct Reference

One execution's place in the attachment graph: who attached it, if anyone.

```
#include <async.hpp>
```

Public Attributes

- `std::weak_ptr< attachment >` `attacher`
The link of the execution that attached this one; empty when nothing has.

8.1.1 Detailed Description

One execution's place in the attachment graph: who attached it, if anyone.

Deliberately at namespace scope rather than nested in [execution](#), so that it is one type rather than one per specialisation. An execution may be attached by an execution of any specialisation, and the whole point of this record is that the chain can be walked without knowing what any link's action type is.

An execution holds one of these by `std::shared_ptr` and it dies with the execution, so it doubles as the liveness token the attached side needs: a link whose weak reference still locks belongs to an execution that is still there.

The documentation for this struct was generated from the following file:

- `async.hpp`

8.2 `untangle::async::execution< actionT >` Class Template Reference

Asynchronous template execution class.

```
#include <async.hpp>
```

Public Member Functions

- [execution](#) ()
Constructs a new execution object with the default name. Call [create_instance\(\)](#) instead.
- [execution](#) (std::string exec_name)
Constructs a new named execution object.
- [~execution](#) ()
Destroys the execution object, once its worker has left.
- bool [is_running](#) () const
Checks if this execution has finished.
- void [stop](#) ()
Stops the worker, after the actions already queued have run.
- template<typename... Args>
void [add_action](#) (actionT action, Args... args)
- template<typename otherT>
void [attach](#) (const std::shared_ptr< otherT > &other)
Attaches another execution, so that this one triggers it.
- template<typename otherT>
bool [detach](#) (otherT &other)
Detaches an execution previously attached by [attach\(\)](#).

Static Public Member Functions

- template<typename... Args>
static std::shared_ptr< execution > [create_instance](#) (Args &&... args)
Creates an execution owned by a std::shared_ptr, which is what binding to it requires.
- template<typename classT, typename T>
static void [bind_action_and_method](#) (actionT &action, const std::shared_ptr< classT > &obj, T classT::*method, const std::shared_ptr< execution > &async_exec)
Binds asynchronously an external action to a class function member.
- template<typename T>
static void [bind_action_and_function](#) (actionT &action, const T &Fn, const std::shared_ptr< execution > &async_exec)
Binds asynchronously an external action to a plain function.

Static Public Attributes

- static constexpr auto default_name = "default_name"
Name of an execution built unnamed.

8.2.1 Detailed Description

```
template<typename actionT>
class untangle::async::execution< actionT >
```

Asynchronous template execution class.

An async execution provides a mechanism to queue actions and execute them sequentially on a separate thread. It may also attach another [execution](#) object and trigger its actions. This way actions of different types may be executed on the same thread.

The mechanism relies on an "asynchronous binding" created by [bind_action_and_method\(\)](#) or [bind_action_and_function\(\)](#).

Template Parameters

actionT	It represents the type of the action. It is specified as <code>std::function<...></code> and should match the signature of the bound function or class method.
---------	--

8.2.2 Constructor & Destructor Documentation

8.2.2.1 execution() [1/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution () [inline]
```

Constructs a new execution object with the default name. Call [create_instance\(\)](#) instead.

The name identifies an execution in the warnings it reports, and is otherwise unused. An execution built without one keeps the default name rather than an empty one, so a warning always names something.

8.2.2.2 execution() [2/2]

```
template<typename actionT>
untangle::async::execution< actionT >::execution (
    std::string exec_name) [inline], [explicit]
```

Constructs a new named execution object.

An execution constructed directly cannot be bound to - [bind_action_and_method\(\)](#) and [bind_action_and_function\(\)](#) require a `std::shared_ptr`. Use [create_instance\(\)](#) for one that is going to carry bound actions.

Parameters

exec_name	- A name for this execution.
-----------	------------------------------

8.2.2.3 ~execution()

```
template<typename actionT>
untangle::async::execution< actionT >::~~execution () [inline]
```

Destroys the execution object, once its worker has left.

The worker is detached and cannot be joined, so this waits on the same "running" state that [execution_poll](#) reports - the only handle a detached worker offers. It returns at once for an execution that was never started, or that the caller has already polled to a stop.

Remarks

It first takes this execution out of whatever attached it, which is the inverse of [attach\(\)](#) and the counterpart of the [execution_poll::remove\(\)](#) below.

This is safe only because running is cleared as the very last thing the worker does. Nothing may be added after it in `execute()` or `loop()`: the object can be freed the moment it reads false.

8.2.3 Member Function Documentation

8.2.3.1 create_instance()

```
template<typename actionT>
template<typename... Args>
std::shared_ptr< execution > untangle::async::execution< actionT >::create_instance (
    Args &&... args) [inline], [static]
```

Creates an execution owned by a `std::shared_ptr`, which is what binding to it requires.

[bind_action_and_method\(\)](#) and [bind_action_and_function\(\)](#) take the execution as a `std::shared_ptr` and hold it weakly, so an execution that is to be bound has to be created this way. One built on the stack, as a data member, or through a bare new is still a usable execution

- actions can be handed to [add_action\(\)](#) directly - it simply cannot be bound, and the attempt does not compile.

The arguments are forwarded to a constructor, so this factory does not have to be revisited when one is added.

Parameters

args	- Constructor arguments: a name, or nothing for the default name.
------	---

Returns

- A `std::shared_ptr` owning the new execution.

8.2.3.2 bind_action_and_method()

```
template<typename actionT>
template<typename classT, typename T>
void untangle::async::execution< actionT >::bind_action_and_method (
    actionT & action,
    const std::shared_ptr< classT > & obj,
    T classT::* method,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

Binds asynchronously an external action to a class function member.

It creates an action as a binding to a class method (by `untangle::bind()`), and assigns to action a callable that passes it to [add_action\(\)](#). The execution is taken as a `std::shared_ptr` and held as a `std::weak_ptr`, exactly as `untangle::bind()` holds the bound object: the action can therefore outlive the execution and report a dead binding rather than following a dangling reference.

Attention

Invoking action after the execution has been destroyed throws `untangle::invalid_action`. `execute↵actions()` catches it, so such an action is dropped with a warning rather than ending the worker thread.

Parameters

action	[in,out] - An action of type <code>std::function<...></code> .
obj	- A <code>std::shared_ptr</code> that wraps the bound class object.
method	- Pointer to function member. It is specified as <code>&<class type>::<function member></code> .
async_exec	- A <code>std::shared_ptr</code> owning the execution the action is queued on. Requiring it here is what keeps an execution that no <code>shared_ptr</code> owns from being bound at all.

8.2.3.3 `bind_action_and_function()`

```
template<typename actionT>
template<typename T>
void untangle::async::execution< actionT >::bind_action_and_function (
    actionT & action,
    const T & Fn,
    const std::shared_ptr< execution< actionT > > & async_exec) [inline], [static]
```

Binds asynchronously an external action to a plain function.

It wraps `Fn` in an action and assigns to `action` a callable that passes it to `add_action()`. The execution is held weakly, for the reason given on `bind_action_and_method()`.

Parameters

action	[in,out] - An action of type <code>std::function<...></code> .
Fn	- A plain function.
async_exec	- A <code>std::shared_ptr</code> owning the execution the action is queued on.

8.2.3.4 `is_running()`

```
template<typename actionT>
bool untangle::async::execution< actionT >::is_running () const [inline]
```

Checks if this execution has finished.

Returns

`true` - The execution has not finished.
`false` - The execution has finished.

8.2.3.5 `stop()`

```
template<typename actionT>
void untangle::async::execution< actionT >::stop () [inline]
```

Stops the worker, after the actions already queued have run.

Remarks

An action passed to `add_action()` after this call is refused, not queued: `stop()` is the end of this execution's working life, and an action accepted then would never run.

8.2.3.6 add_action()

```
template<typename actionT>
template<typename... Args>
void untangle::async::execution< actionT >::add_action (
    actionT action,
    Args... args) [inline]
```

Template Parameters

Args	
------	--

Parameters

action	
args	

8.2.3.7 attach()

```
template<typename actionT>
template<typename otherT>
void untangle::async::execution< actionT >::attach (
    const std::shared_ptr< otherT > & other) [inline]
```

Attaches another execution, so that this one triggers it.

The attached execution's pending actions are run by this one's worker, and stopping this one stops that one too.

The actuators are given other's own actions, so the attachment is a pointer into other. What makes that safe is the record left on the other side: other is told which actuators hold it, and [~execution\(\)](#) takes itself back out of them. The attacher's lifetime token guards those pointers, so an attached execution outliving its attacher follows nothing.

Parameters

other	- A <code>std::shared_ptr</code> owning the execution to trigger. Requiring it here is what keeps an execution that no <code>std::shared_ptr</code> owns from being attached at all, exactly as bind_action_and_method() requires one to bind. Attaching the same execution twice adds it twice; a single detach() still removes it completely, because <code>untangle::actuator::remove()</code> erases every match.
-------	---

Exceptions

invalid_attachment	- if other is attached already. An execution has at most one attacher, which is what keeps the attachment graph a forest and the cycle check below a walk rather than a search.
invalid_attachment	- if the attachment would close a cycle, which includes attaching an execution to itself. Driving a cycle recurses until the stack is gone, so it is refused here, where the caller still has a stack to be told on.

8.2.3.8 detach()

```
template<typename actionT>
template<typename otherT>
bool untangle::async::execution< actionT >::detach (
    otherT & other) [inline]
```

Detaches an execution previously attached by [attach\(\)](#).

Both paths are unwired, the triggering one and the stopping one: an attachment that could only be half undone would leave this execution still able to stop one it no longer drives.

Parameters

other	- The execution to stop triggering. Detaching one that was never attached is not an error: the caller asked for a state that already holds.
-----------------------	---

Returns

true - other was attached and is no longer.
false - other was not attached to this execution.

The documentation for this class was generated from the following file:

- [async.hpp](#)

8.3 untangle::async::execution_poll Class Reference

Execution poll class.

```
#include <async.hpp>
```

Public Member Functions

- `template<typename asyncexecT>`
`void add (asyncexecT &async_exec)`
 Adds an [execution](#) object to the poll.
- `template<typename asyncexecT>`
`void remove (asyncexecT &async_exec)`
 Removes an [execution](#) object from the poll.
- `auto is_running ()`
 Checks if the execution objects are running.

Static Public Member Functions

- `static execution_poll & get ()`
 Gets the single-tone instance.

8.3.1 Detailed Description

Execution poll class.

A single-tone class that may be used to verify if the polled [execution](#) objects have finished their processing. If at least one [execution](#) object is running then this poll has the "running" state.

Remarks

An [execution](#) object runs in a detached thread, hence it is not joinable. The caller can wait for an [execution](#) to be finished only by checking the "running" state([execution::is_running\(\)](#)). This class represents a convenient way to check any number of [execution](#) objects.

8.3.2 Member Function Documentation

8.3.2.1 add()

```
template<typename asyncexecT>
void untangle::async::execution_poll::add (
    asyncexecT & async_exec) [inline]
```

Adds an [execution](#) object to the poll.

Parameters

<code>async_exec</code>	An execution object.
-------------------------	--------------------------------------

8.3.2.2 remove()

```
template<typename asyncexecT>
void untangle::async::execution_poll::remove (
    asyncexecT & async_exec) [inline]
```

Removes an [execution](#) object from the poll.

Parameters

async_exec	An execution object.
------------	--------------------------------------

8.3.2.3 is_running()

```
auto untangle::async::execution_poll::is_running () [inline]
```

Checks if the execution objects are running.

Returns

true - if at least one [execution](#) object in this poll is running.

8.3.2.4 get()

```
execution_poll & untangle::async::execution_poll::get () [inline], [static]
```

Gets the single-tone instance.

Returns

[execution_poll](#) An [execution_poll](#) instance.

The documentation for this class was generated from the following file:

- [async.hpp](#)

8.4 untangle::async::invalid_attachment Struct Reference

Invalid attachment exception.

```
#include <async.hpp>
```

Public Member Functions

- [invalid_attachment](#) (std::string text)
Construct a new invalid attachment object.
- const char * [what](#) () const noexcept override
The message text describing the reason of this exception.

8.4.1 Detailed Description

Invalid attachment exception.

Remarks

Raised by `execution::attach()` when the attachment asked for would close a cycle - either between two executions that each attach the other, or an execution attached to itself. Driving such a cycle recurses until the stack is gone, so it is refused where the caller can still be told about it.

Deliberately not an `untangle::invalid_action`. That one reports a binding whose target has died, and `execution::execute_actions()` swallows it by design; a cycle is a caller error and must not be swallowed.

8.4.2 Constructor & Destructor Documentation

8.4.2.1 `invalid_attachment()`

```
untangle::async::invalid_attachment::invalid_attachment (
    std::string text) [inline], [explicit]
```

Construct a new invalid attachment object.

Parameters

text	- A message text, describing the reason of this exception.
------	--

8.4.3 Member Function Documentation

8.4.3.1 `what()`

```
const char * untangle::async::invalid_attachment::what () const [inline], [override], [noexcept]
```

The message text describing the reason of this exception.

Returns

const char* - The message text.

The documentation for this struct was generated from the following file:

- `async.hpp`

Chapter 9

File Documentation

9.1 async.hpp

```
00001 // Copyright (c) 2018 Nicolae Popescu. MIT License.
00002
00006 #pragma once
00007
00008 #include <actuator/actuator.hpp>
00009 #include <algorithm>
00010 #include <atomic>
00011 #include <condition_variable>
00012 #include <exception>
00013 #include <memory>
00014 #include <mutex>
00015 #include <print>
00016 #include <string>
00017 #include <thread>
00018
00019 namespace untangle {
00020 template <typename actionT>
00021 struct actuator;
00022
00023 namespace async {
00024 template <typename actionT>
00025 class execution;
00026 }
00027 } // namespace untangle
00028
00029 namespace untangle {
00030 namespace async {
00034
00047 struct invalid_attachment : std::exception {
00053     explicit invalid_attachment(std::string text) : message(std::move(text)) {}
00054
00060     const char* what() const noexcept override { return message.c_str(); }
00061
00062 private:
00063     std::string message;
00064 };
00065
00078 struct attachment {
00080     std::weak_ptr<attachment> attacher;
00081 };
00082
00097 class execution_poll {
00098 public:
00104     template <typename asyncexecT>
00105     void add(asyncexecT& async_exec) {
00106         std::lock_guard<std::mutex> lock(actuator_mutex);
00107
00108         if (!actuator_is_running.is_connected()) {
00109             actuator_is_running = untangle::connect(async_exec.action_is_running);
00110         } else {
00111             actuator_is_running.add(&async_exec.action_is_running);
00112         }
00113     }
00114
00123     template <typename asyncexecT>
00124     void remove(asyncexecT& async_exec) {
00125         std::lock_guard<std::mutex> lock(actuator_mutex);
```

```

00126
00127     actuator_is_running.remove(&async_exec.action_is_running);
00128 }
00129
00130 auto is_running() {
00131     std::lock_guard<std::mutex> lock(actuator_mutex);
00132     actuator_is_running();
00133
00134     auto result = false;
00135     for (const auto& ret : actuator_is_running.results) {
00136         result |= ret;
00137     }
00138     return result;
00139 }
00140
00141 static execution_poll& get() {
00142     static execution_poll instance;
00143     return instance;
00144 }
00145
00146 private:
00147     execution_poll() = default;
00148     ~execution_poll() = default;
00149     actuator<std::function<bool(void)>> actuator_is_running;
00150     mutable std::mutex actuator_mutex;
00151 };
00152
00153 template <typename actionT>
00154 class execution {
00155     // An execution may attach one of a different specialisation - the smoke test attaches an
00156     // execution<function<void(int)>> to an execution<function<void(void)>> - and that is a different
00157     // class with no access to this one's members. attach() and detach() need to read and write
00158     // attachment_lifetime on it.
00159     template <typename otherActionT>
00160     friend class execution;
00161
00162 public:
00163     template <typename... Args>
00164     static std::shared_ptr<execution> create_instance(Args&&... args) {
00165         return std::make_shared<execution>(std::forward<Args>(args)...);
00166     }
00167
00168     execution() : execution(std::string(default_name)) {}
00169
00170     explicit execution(std::string exec_name) : name(std::move(exec_name)) {
00171         other_this = this;
00172         action_execute = untangle::bind(other_this, &execution<actionT>::execute_actions);
00173         action_stop = untangle::bind(other_this, &execution<actionT>::stop);
00174         action_is_running = untangle::bind(other_this, &execution<actionT>::is_running);
00175     }
00176
00177     ~execution() {
00178         // Out of the attacher first, before the worker is even asked to stop: from here on nothing
00179         // may reach this object, and the actions about to be destroyed are the ones it holds.
00180         if (!attachment_lifetime->attacher.expired()) {
00181             attacher_execute->remove(&action_execute);
00182             attacher_stop->remove(&action_stop);
00183         }
00184
00185         {
00186             std::lock_guard<std::mutex> lock(action_mutex);
00187             started = false;
00188             stopped = true;
00189         }
00190
00191         action_cv.notify_all();
00192
00193         while (running.load()) { // time of check
00194             std::this_thread::sleep_for(std::chrono::milliseconds(1));
00195         }
00196
00197         // time of use: this object is freed once the destructor returns, so the check above is only
00198         // safe because running is the last thing the worker touches
00199         execution_poll::get().remove(*this);
00200     }
00201
00202     template <typename classT, typename T>
00203     static void bind_action_and_method(actionT& action, const std::shared_ptr<classT>& obj,
00204                                         T classT::* method,
00205                                         const std::shared_ptr<execution>& async_exec) {
00206         actionT async_action = untangle::bind(obj, method);
00207
00208         action = [wp = std::weak_ptr<execution>(async_exec),
00209                 async_action](auto... args) -> actionT::result_type {
00210             // lock() also keeps the execution alive for the duration of the call
00211             const auto exec = wp.lock();
00212             if (!exec) {

```

```

00303         throw invalid_action("bind: invalid execution");
00304     }
00305     exec->add_action(async_action, args...);
00306     return typename actionT::result_type();
00307 };
00308 }
00309
00321 template <typename T>
00322 static void bind_action_and_function(actionT& action, const T& Fn,
00323                                     const std::shared_ptr<execution>& async_exec) {
00324     actionT async_action = Fn;
00325
00326     action = [wp = std::weak_ptr<execution>(async_exec),
00327              async_action](auto... args) -> actionT::result_type {
00328         const auto exec = wp.lock();
00329         if (!exec) {
00330             throw invalid_action("bind: invalid execution");
00331         }
00332         exec->add_action(async_action, args...);
00333         return typename actionT::result_type();
00334     };
00335 }
00336
00343 bool is_running() const { return running.load(); }
00344
00345 void run() {
00346     running = true;
00347     this_thread = std::thread(&execution::execute, this);
00348     this_thread.detach();
00349 }
00350
00351 void start() {
00352     running = true;
00353     {
00354         std::lock_guard<std::mutex> lock(action_mutex);
00355         started = true;
00356         stopped = false;
00357     }
00358
00359     this_thread = std::thread(&execution::loop, this);
00360     this_thread.detach();
00361 }
00362
00369 void stop() {
00370     {
00371         std::lock_guard<std::mutex> lock(action_mutex);
00372         started = false;
00373         stopped = true;
00374     }
00375
00376     // The worker drains what is already queued before it leaves loop(), so stop() does not have to
00377     // spin on the list; waking it is enough.
00378     action_cv.notify_all();
00379
00380     if (actuator_stop.is_connected()) {
00381         actuator_stop();
00382     }
00383 }
00384
00392 template <typename... Args>
00393 void add_action(actionT action, Args... args) {
00394     {
00395         std::lock_guard<std::mutex> lock(action_mutex);
00396
00397         // Once stopped, the worker is on its way out and would never reach this action; dropping it
00398         // here is what keeps it from sitting in the list looking as though it were pending.
00399         if (stopped) {
00400             std::println(stderr, "warning: execution '{}' is stopped, action not added", name);
00401             return;
00402         }
00403
00404         action_list.push_back(std::bind(action, args...));
00405     }
00406
00407     action_cv.notify_one();
00408 }
00409
00435 template <typename otherT>
00436 void attach(const std::shared_ptr<otherT>& other) {
00437     if (static_cast<const void*>(other.get()) == static_cast<const void*>(this)) {
00438         throw invalid_attachment("attach: an execution cannot be attached to itself");
00439     }
00440
00441     if (!other->attachment_lifetime->attacher.expired()) {
00442         throw invalid_attachment("attach: that execution is attached already");
00443     }
00444 }

```

```

00445 // Refusing an execution that has an attacher already leaves every execution with at most one,
00446 // so the graph is a forest and this new edge closes a cycle exactly when other is somewhere up
00447 // this execution's own chain of attachers. Walking it is the whole check.
00448 for (auto link = attachment_lifetime->attacher.lock(); link; link = link->attacher.lock()) {
00449     if (link == other->attachment_lifetime) {
00450         throw invalid_attachment("attach: the attachment would close a cycle");
00451     }
00452 }
00453
00454 if (!actuator_execute.is_connected()) {
00455     actuator_execute = untangle::connect(other->action_execute);
00456 } else {
00457     actuator_execute.add(&other->action_execute);
00458 }
00459
00460 if (!actuator_stop.is_connected()) {
00461     actuator_stop = untangle::connect(other->action_stop);
00462 } else {
00463     actuator_stop.add(&other->action_stop);
00464 }
00465
00466 // What ~execution() needs to take itself back out of these actuators: the link proves this
00467 // attacher is still there, and the two pointers are only ever followed while it does.
00468 other->attachment_lifetime->attacher = attachment_lifetime;
00469 other->attacher_execute = &actuator_execute;
00470 other->attacher_stop = &actuator_stop;
00471 }
00472
00485 template <typename otherT>
00486 bool detach(otherT& other) {
00487     // The actuator's own action list is the record of what is attached; nothing else has to keep
00488     // one. An action is stored as a pointer, so the attachment is found by identity.
00489     const auto& actions = actuator_execute.actions;
00490     if (std::find(actions.begin(), actions.end(), &other.action_execute) == actions.end()) {
00491         return false;
00492     }
00493
00494     actuator_execute.remove(&other.action_execute);
00495     actuator_stop.remove(&other.action_stop);
00496
00497     other.attachment_lifetime->attacher.reset();
00498     other.attacher_execute = nullptr;
00499     other.attacher_stop = nullptr;
00500
00501     return true;
00502 }
00503
00504 auto result() { return _result; }
00505
00506 static constexpr auto default_name = "default_name";
00507
00508 std::function<void(void)> action_execute;
00509 std::function<void(void)> action_stop;
00510 std::function<bool(void)> action_is_running;
00511 std::function<void(void)> on_finished;
00512 std::string name = default_name;
00513
00514 private:
00515     using queued_action_t = std::function<typename actionT::result_type(void)>;
00516
00520     void execute_action(queued_action_t& action) {
00521         if constexpr (std::is_void_v<typename actionT::result_type>) {
00522             action();
00523         } else {
00524             _result = action();
00525         }
00526     }
00527
00528     void execute_actions() {
00529         // The action is taken off the list under the lock and invoked with the lock released: an action
00530         // is caller code that may run for a while, and may itself call add_action().
00531         for (;;) {
00532             queued_action_t action;
00533
00534             {
00535                 std::lock_guard<std::mutex> lock(action_mutex);
00536                 if (action_list.empty()) {
00537                     break;
00538                 }
00539
00540                 action = std::move(action_list.front());
00541                 action_list.pop_front();
00542             }
00543
00544             // An action bound to an object that has since died throws invalid_action. Letting it leave a
00545             // thread function calls std::terminate, and a detached worker gives no one a chance to catch
00546             // it; the actuator drops such an action, and so does this.

```

```

00547     try {
00548         execute_action(action);
00549     } catch (const invalid_action& ia) {
00550         std::println(stderr, "warning: execution '{}' dropped an invalid action: {}", name,
00551             ia.what());
00552     }
00553 }
00554
00555 if (actuator_execute.is_connected()) {
00556     actuator_execute();
00557 }
00558 }
00559
00560 void execute() {
00561     execute_actions();
00562
00563     if (on_finished) {
00564         on_finished();
00565     }
00566
00567     std::cout << "finishing thread" << std::endl;
00568
00569     running = false;
00570 }
00571
00572 void loop() {
00573     for (;;) {
00574         {
00575             std::unique_lock<std::mutex> lock(action_mutex);
00576
00577             // A bounded wait rather than a plain one: an attached execution has its own list and no way
00578             // to notify this condition variable, so the worker still has to look in on it periodically.
00579             action_cv.wait_for(lock, std::chrono::milliseconds(10),
00580                 [this] { return !action_list.empty() || !started; });
00581
00582             if (!started && action_list.empty()) {
00583                 break;
00584             }
00585         }
00586
00587         execute_actions();
00588     }
00589
00590     // What was queued before stop() still belongs to this execution; nothing can have been added
00591     // after it, because add_action() refuses once stopped.
00592     execute_actions();
00593
00594     std::cout << "thread finished" << std::endl;
00595
00596     // Must stay last: ~execution() may free this object the moment it reads false.
00597     running = false;
00598 }
00599
00600 std::thread this_thread;
00601
00602 std::atomic_bool started = {false};
00603 std::atomic_bool running = {false};
00604
00605 // Set by stop(), and the difference between "not started yet" and "finished for good": actions
00606 // may be queued before run()/start(), but not after stop().
00607 std::atomic_bool stopped = {false};
00608
00609 // action_list, started and stopped are written by every thread that calls add_action() or stop()
00610 // and read by the worker; nothing touches them outside this mutex. action_cv is what replaced the
00611 // spin in loop() and in stop().
00612 mutable std::mutex action_mutex;
00613 std::condition_variable action_cv;
00614
00615 std::list<std::function<typename actionT::result_type(void)>> action_list;
00616
00625 using void_actuator = actuator<std::function<void(void)>>;
00626
00627 void_actuator actuator_execute;
00628 void_actuator actuator_stop;
00629
00643 std::shared_ptr<attachment> attachment_lifetime = std::make_shared<attachment>();
00644
00658 void_actuator* attacher_execute = nullptr;
00659 void_actuator* attacher_stop = nullptr;
00660
00661 // std::vector cannot hold void type; use an arbitrary type e.g. int
00662 using resultT = std::conditional<std::is_void<typename actionT::result_type>::value, int,
00663     typename actionT::result_type>;
00664 resultT::type _result;
00665
00666 execution* other_this;
00667 };

```

```
00668 } // namespace async
00669 } // namespace untangle
```

Index

- ~execution
 - untangle::async::execution< actionT >, [19](#)
- add
 - untangle::async::execution_poll, [24](#)
- add_action
 - untangle::async::execution< actionT >, [21](#)
- async, [1](#)
- attach
 - untangle::async::execution< actionT >, [22](#)
- bind_action_and_function
 - untangle::async::execution< actionT >, [21](#)
- bind_action_and_method
 - untangle::async::execution< actionT >, [20](#)
- create_instance
 - untangle::async::execution< actionT >, [20](#)
- detach
 - untangle::async::execution< actionT >, [23](#)
- execution
 - untangle::async::execution< actionT >, [19](#)
- get
 - untangle::async::execution_poll, [25](#)
- invalid_attachment
 - untangle::async::invalid_attachment, [26](#)
- is_running
 - untangle::async::execution< actionT >, [21](#)
 - untangle::async::execution_poll, [25](#)
- namespace untangle: functions, [13](#)
- remove
 - untangle::async::execution_poll, [24](#)
- stop
 - untangle::async::execution< actionT >, [21](#)
- untangle, [15](#)
- untangle::async::attachment, [17](#)
- untangle::async::execution< actionT >, [17](#)
 - ~execution, [19](#)
 - add_action, [21](#)
 - attach, [22](#)
 - bind_action_and_function, [21](#)
 - bind_action_and_method, [20](#)
 - create_instance, [20](#)
 - detach, [23](#)
 - execution, [19](#)
 - is_running, [21](#)
 - stop, [21](#)
- untangle::async::execution_poll, [23](#)
 - add, [24](#)
 - get, [25](#)
 - is_running, [25](#)
 - remove, [24](#)
- untangle::async::invalid_attachment, [25](#)
 - invalid_attachment, [26](#)
 - what, [26](#)
- what
 - untangle::async::invalid_attachment, [26](#)